

SMC Boot ROM — Functional Specification

SMC Boot ROM — Functional Specification

Document: `01_BOOTROM_Specification.md` **Module:** `smc::bootrom` (SystemC/TLM-2.0 Loosely-Timed model, CCI-parameterised) **IP family:** SMC IP library (sister to `smc::plic`, `smc::clint`) **Spec base:** `tt-oca-hw/meta/registers/rdl/sep_boot_rom.rdl` (authoritative read-only mem-block contract; see §1) **Status:** Frozen for SMC bring-up; tracks the cocotb conformance suite under `tt-oca-hw/dv/oss/shims/sep/memories/tests/conformance/` **Companion docs:** - `02_BOOTROM_LowLevel_Design.md` — internal SystemC implementation - `03_BOOTROM_Test_Plan.md` — verification strategy & test list

Contents

1. Purpose & scope
 2. Conformance & references
 3. Feature summary
 4. Configuration parameters
 5. Block diagram & port list
 6. Theory of operation
 7. Memory map
 8. Functional behaviour
 9. Reset behaviour
 10. Bus interface (TLM-2.0 / AXI4-style)
 11. Error handling
 12. Preload formats
 13. Compliance matrix
 14. Revision history
 15. Glossary
-

1. Purpose & scope

The **SMC Boot ROM** is a non-volatile, preloaded, read-only memory in the System Management Controller (SMC) IP library. It holds the immutable first-stage boot loader (and the public root-of-trust signature roots) that the SMC Rocket RV64GC CPU cluster fetches after cold reset — it is the only SMC region addressable before the alias / M-mode remap is programmed (see `doc/01_SMC_Architecture.md` §6). It is functionally trivial — software cannot modify it — but it is on the critical bring-up path: a bug in the preload pipeline manifests as “the core refuses to boot”, which is one of the hardest classes of platform failure to debug in simulation.

The model is a **generic, role-agnostic read-only mem block**. The same `smc::bootrom` is instantiated wherever a preloaded ROM is needed: the SMC CPU-cluster boot ROM, and — because the two are structurally identical — the SEP (Secure Enclave Processor) boot ROM. Everything role-specific (size,

contents, base address) is supplied through CCI presets, so a single model serves every boot-ROM instance on the platform.

Because there is no separate `smc_boot_rom.rdl`, this model is **specified and validated against the authoritative read-only mem contract that does exist** — `sep_boot_rom.rdl` and its cocotb conformance suite. That RDL declares a plain 64-bit `mem` with `sw=r`, `hw=r`; the contract is identical for any SMC-fabric boot ROM, which is exactly why one model covers both roles.

This specification defines the **externally-observable behaviour** of the SMC Boot ROM SystemC model: configuration, memory map, access semantics, error responses, reset, and bus contract. It is the contract that:

- The CPU cluster fetches its very first instruction against,
- The cocotb conformance suite measures, and
- Any future RTL or platform integration treats as authoritative.

Internal implementation choices (data layout, preload parser, the CCI-parameter scaffolding, single-driver discipline) are documented in `02_BOOTROM_LowLevel_Design.md`.

2. Conformance & references

Source	Authority
<code>tt-oca-hw/meta/registers/rdl/sep_boot_rom.rdl</code>	Ground-truth memory map (<code>mem</code> -type, 64-bit wide, <code>sw=r</code> , <code>hw=r</code>)
<code>tt-oca-hw/dv/oss/shims/sep/memories/tests/conformance/test_sep_boot_rom_rw.py</code>	Read/write conformance claims C2–C4
<code>tt-oca-hw/dv/oss/shims/sep/memories/tests/conformance/test_sep_boot_rom_preload.py</code>	Preload-format conformance
<code>tt-oca-hw/hw/smc/data/scripts/bootrom.rv64.img</code>	Sample binary preload image (raw little-endian 8-byte words)
<code>tt-oca-hw/dv/smc/tb/meta/scripts/bootrom_sanity.rv64.hex</code>	Sample hex preload (one 64- bit big-endian ASCII word per line)
<code>01_PLIC_Specification.md</code> , <code>01_CLINT_Specification.md</code>	Sister SMC IPs — same conventions

The SystemC model is **byte-identical** to the RDL memory layout: a contiguous `NUM_ENTRIES`-by-8-byte memory at offset 0, addressable at any naturally-aligned `{1, 2, 4, 8}`-byte width.

3. Feature summary

Feature	Value / behaviour
Standard	OCA SMC boot ROM; structurally identical to a <code>mem</code> block in RDL (SEP-compatible)
Size	64 KiB (default; configurable via CCI <code>size_bytes</code>)
Native word width	64 bits (matches <code>regwidth</code> = <code>memwidth</code> = 64 in the RDL)
Supported access widths	1, 2, 4, 8 bytes (naturally aligned)
Software write semantics	Silently ignored — <code>TLM_OK_RESPONSE</code> , contents unchanged
Read-after-write	Returns the unchanged preloaded value (never the attempted write)
Preload formats	Hex (text, big-endian) or binary (raw little-endian)
Zero-init behaviour	When <code>init_file</code> is empty: all bytes read as <code>0x00</code>
Bus interface	TLM-2.0 LT target socket (AXI4-style)
DMI	Not granted (kept consistent with other SMC IPs; future work)
Reset polarity	Active-low, synchronous; structural no-op (ROM has no state to clear)
Configuration interface	SystemC CCI 1.0 (<code>cci::cci_param<T></code>)
Annotated TLM delay	1.0 ns default; mutable via the <code>access_delay_ns</code> CCI param

4. Configuration parameters

The model exposes **four real `cci::cci_param<T>` declarations**. The size and preload parameters are immutable (changing them after elaboration would invalidate the in-memory image); the access delay is mutable so simulations can sweep bus speeds without rebuilding.

Parameter	Type	Default	Mutability	Notes
<code>size_bytes</code>	<code>uint64_t</code>	<code>0x10000</code>	immutable	Must be a non-zero multiple of 8 (the native 64-bit word width).
<code>init_file</code>	<code>std::string</code>	<code>""</code>	immutable	Path to preload image; empty = zero-fill (<code>test_preload_zero_init</code> claim).
<code>init_file_format</code>	<code>std::string</code>	<code>"auto"</code>	immutable	<code>"hex"</code> , <code>"bin"</code> , or <code>"auto"</code> (pick <code>bin</code> for <code>.img/.bin</code> , else <code>hex</code>).
<code>access_delay_ns</code>	<code>double</code>	<code>1.0</code>	mutable	TLM <code>b_transport</code> annotated delay. Claim C2 (1-cycle <code>rvalid</code>) $\Rightarrow \approx 1$ ns.

Override before construction via the CCI broker:

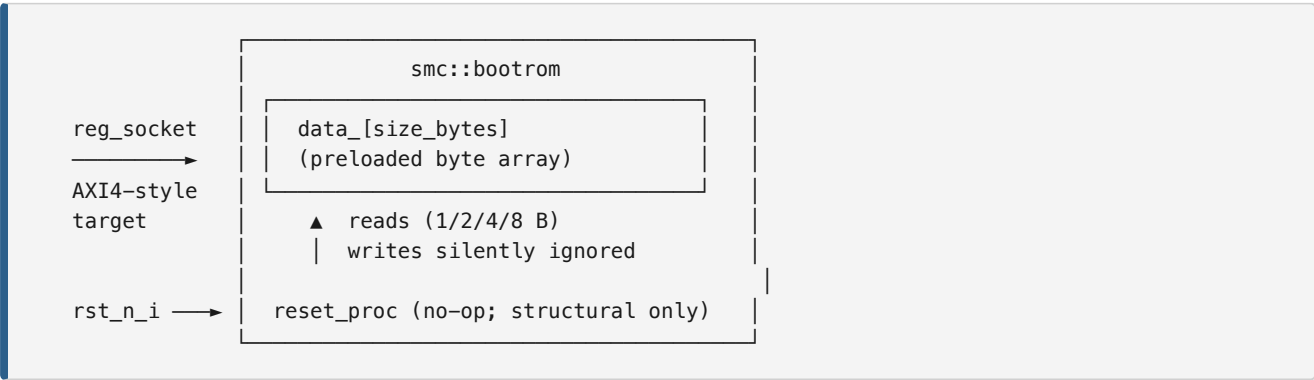
```
broker.set_preset_cci_value("smc.bootrom.size_bytes",
    cci::cci_value(uint64_t(0x10000)));
broker.set_preset_cci_value("smc.bootrom.init_file",
    cci::cci_value(std::string("bootrom.hex")));
broker.set_preset_cci_value("smc.bootrom.init_file_format",
    cci::cci_value(std::string("hex")));
broker.set_preset_cci_value("smc.bootrom.access_delay_ns",
    cci::cci_value(2.0));
```

The constructor still accepts a `bootrom_cfg` struct; its fields supply **defaults** for the corresponding CCI params, so old code that does not use the CCI broker continues to work unchanged.

Fixed structural constants:

Constant	Value	Description
WORD_BYTES	8	Native ROM word width (from RDL memwidth = 64)
MAX_ACCESS_BYTES	8	Largest single b_transport access width accepted

5. Block diagram & port list



Port	Direction	Type	Description
reg_socket	target	tlm_utils::simple_target_socket<bootrom>	TLM-2.0 target socket; AXI4-style read access
rst_n_i	input	sc_in<bool>	Active-low synchronous reset (no-op; kept for IP-suite symmetry)

The Boot ROM has **no output ports**. It is a passive memory.

6. Theory of operation

The Boot ROM is the simplest possible target IP. It owns a single contiguous byte array (`size_bytes` long, default 64 KiB) loaded from a preload file at elaboration time, and serves `b_transport` reads verbatim from that array. Writes — both production (`b_transport`) and debug (`transport_dbg`) — are silently discarded; the contents cannot be mutated through the bus, exactly as the underlying RDL declares (`sw=r, hw=r`).

Three properties make this trivial-looking IP non-trivial in practice:

1. **Preload format coupling.** The SMC bring-up flow uses two different preload formats — raw binary `.img` and ASCII hex `.hex` — depending on which point in the pipeline the image was emitted. Both must be supported, and both must produce **byte-identical** in-memory images so the fetching core sees the same instruction bytes regardless of which format the integrator chose.
2. **Sparse-non-zero overflow tolerance.** Hex preload files generated by the SMC tooling are often sized for a larger ROM than the one they are loaded into; the tail bytes are zero-padded. The model accepts a longer-than-`size_bytes` hex file **iff** the overflow region is entirely zero — the equivalent of `xxd -r -p` truncating to the destination size. A non-zero overflow word is flagged as a configuration error (`SC_REPORT_FATAL`) because it indicates the image was sized for a different platform.
3. **Write-ignore vs. write-error.** The cocotb conformance test `test_rom_write_ignored` asserts that a write to the ROM must **return `rvalid (TLM_OK_RESPONSE)`** and leave the contents unchanged. Returning a bus error on a write would be incorrect — it would deadlock the CPU-cluster boot path. The model therefore distinguishes write-ignore (in-range writes; OK + no-mutation) from real bus errors (out-of-window, misaligned, unsupported width).

7. Memory map

All offsets are byte offsets relative to the Boot ROM base address.

Range	Region	Access	Size
<code>0x00000 .. size_bytes - 1</code>	<code>mem_array[0..N - 1]</code>	R+W*	8 B × N (default N = 0x2000)
<code>≥ size_bytes</code>	Out-of-window	—	TLM_ADDRESS_ERROR_RESPONSE

*Write access is silently ignored (W = “write request accepted; data discarded”). See §8.2.

The default `size_bytes = 0x10000` matches the RDL declaration `NUM_ENTRIES = 0x2000` (8192 entries × 8 bytes = 64 KiB).

8. Functional behaviour

8.1 Reads

A read at byte offset `0` with width `W ∈ {1, 2, 4, 8}` returns the byte slice `data_[0..0+W - 1]`. Multi-byte reads honour native CPU endianness — the model never byte-swaps; the preload format determines whether the on-disk image is little- or big-endian (see §12).

Property	Behaviour
Alignment	Naturally aligned to <code>W</code> bytes
Width range	$W \in \{1, 2, 4, 8\}$
Response status	<code>TLM_OK_RESPONSE</code> on success
Annotated delay	$\pm \text{access_delay_ns} \times \text{ns}$ (claim C2)
Side effects	None — reads never mutate state

8.2 Writes

A write at byte offset `0` with width `W` and in-range $0 + W \leq \text{size_bytes}$:

Property	Behaviour
Effect on <code>data_</code>	None (silently discarded)
Response status	<code>TLM_OK_RESPONSE</code> (claim C4)
Annotated delay	$\pm \text{access_delay_ns} \times \text{ns}$
Subsequent read	Returns the unchanged preloaded value

This is the only IP in the SMC suite whose write path is a no-op-by- design rather than an error path; the firmware driver and the SMC fabric must see `rvalid` so the bus does not stall.

8.3 Preload contract

At construction time, if `init_file` is non-empty, the file is opened and decoded according to the resolved format (see §12). The decoded bytes are stored starting at offset 0; any remaining bytes are left at their initial value (zero). If `init_file` is empty the entire ROM is zero-initialised — matching the cocotb `test_preload_zero_init` claim.

Preload errors (file not found, unreadable, malformed) are reported with `SC_REPORT_FATAL` at elaboration time, so an integration mistake fails fast rather than producing silent corruption at run time.

8.4 Reset behaviour

Reset is a **structural no-op**: the ROM holds no mutable state, so an assertion of `rst_n_i` does not alter `data_`. The `reset_proc` SC_METHOD is still registered for IP-suite symmetry — every other SMC IP exposes a `rst_n_i`, so removing it here would break uniform binding patterns at the platform top level.

A read immediately after reset returns the preloaded value, as the cocotb `test_read_after_reset` claim mandates.

9. Reset behaviour

Signal	Polarity	Edge	Effect on ROM state
<code>rst_n_i</code>	active-low	level-low	None (ROM is immutable)

A wired-low `rst_n_i` does not stall the bus interface — reads continue to return preloaded data throughout the reset window. The cocotb conformance harness exploits this by issuing the very first read immediately after the reset de-asserts and expecting valid data on the next cycle.

10. Bus interface (TLM-2.0 / AXI4-style)

10.1 Socket type

```
tlm_utils::simple_target_socket<bootrom> reg_socket;
```

Registered callbacks:

```
reg_socket.register_b_transport (this, &bootrom::b_transport);
reg_socket.register_transport_dbg(this, &bootrom::transport_dbg);
```

DMI is **deliberately not registered**. A read-only ROM is a perfect DMI candidate; we keep it un-granted for parity with the other SMC IPs (see `02_BOOTROM_LowLevel_Design.md §16` for the future-work item).

10.2 Transaction contract

Field	Required value	Notes
command	TLM_READ_COMMAND / TLM_WRITE_COMMAND	TLM_IGNORE_COMMAND → TLM_COMMAND_ERROR_RESPONSE
address	4-byte aligned to data_length, < size_bytes	Otherwise error (see §11)
data_length	1, 2, 4, or 8	Otherwise TLM_BURST_ERROR_RESPONSE
streaming_width	== data_length	Otherwise TLM_BURST_ERROR_RESPONSE
byte_enable_ptr	nullptr (or byte_enable_length == 0)	Non-null → TLM_BYTE_ENABLE_ERROR_RESPONSE
dmi_allowed	(cleared on every transaction)	DMI never granted

10.3 Timing annotation

`b_transport` adds `sc_time(access_delay_ns_p.get_value(), SC_NS)` to the supplied `delay` reference. The model re-reads the CCI param on **every** transaction, so a broker-driven mutation takes effect on the next access.

`transport_dbg` adds nothing — debug accesses are zero-time by TLM-2.0 convention.

The Boot ROM never initiates traffic; it has no quantum keeper and participates in whatever quantum the caller is running.

10.4 transport_dbg

Symmetric to `b_transport` for the validation path. Debug **reads** copy from `data_` exactly like production reads; debug **writes** are silently discarded (same contract as `b_transport`). On validation error `transport_dbg` returns 0; on success it returns the number of bytes transferred (TLM-2.0 convention).

11. Error handling

Condition	Detection	Response
<code>data_length ∉ {1, 2, 4, 8}</code>	<code>b_transport</code> precheck	<code>TLM_BURST_ERROR_RESPONSE</code>
Address not naturally aligned to <code>data_length</code>	<code>b_transport</code> precheck	<code>TLM_BURST_ERROR_RESPONSE</code>
<code>streaming_width ≠ data_length</code>	<code>b_transport</code> precheck	<code>TLM_BURST_ERROR_RESPONSE</code>
Non-null <code>byte_enable_ptr</code>	<code>b_transport</code> precheck	<code>TLM_BYTE_ENABLE_ERROR_RESPONSE</code>
<code>address ≥ size_bytes</code> or <code>address + len > size_bytes</code>	<code>b_transport</code> window check	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
Unsupported command (<code>TLM_IGNORE_COMMAND</code>)	<code>b_transport</code> switch	<code>TLM_COMMAND_ERROR_RESPONSE</code>
Construction with <code>size_bytes = 0</code> or non-multiple of 8	Constructor	<code>SC_REPORT_FATAL</code> at elaboration
Construction with <code>access_delay_ns < 0</code>	Constructor	<code>SC_REPORT_FATAL</code> at elaboration
Preload file missing / unreadable	<code>load_preload()</code>	<code>SC_REPORT_FATAL</code> at elaboration
Malformed hex line	<code>parse_hex_line()</code>	<code>SC_REPORT_FATAL</code> (file + line no.)
Hex preload overflow with non-zero data	<code>load_preload()</code>	<code>SC_REPORT_FATAL</code>
Hex preload overflow with all-zero tail	<code>load_preload()</code>	Tolerated (truncated silently)
In-window write	<code>b_transport</code>	<code>TLM_OK_RESPONSE</code> , contents unchanged (claim C4)

`SC_REPORT_FATAL` is reserved for elaboration-time misconfiguration; runtime failures use TLM response codes so the surrounding fabric and test bench see deterministic bus-level errors rather than simulator aborts.

12. Preload formats

12.1 Hex format (`init_file_format = "hex"`)

Plain text, one 64-bit word per line, big-endian ASCII hex (most significant nibble first). Compatible with `xxd -r -p` round-tripping.

```
# Lines starting with '#' are comments.
# Blank lines are ignored.
000005177c105073      # word 0 - written to data_[0..7] LSB-first
3055107301450513      # word 1 - written to data_[8..15] LSB-first
```

- Each line yields one 8-byte chunk stored starting at the next consecutive 8-byte slot (word 0 → offset 0, word 1 → offset 8, ...).

- The 64-bit value is stored **little-endian** in `data_`, matching the on-disk layout of the binary format below (so the same image produces identical in-memory bytes regardless of which preload format was used).
- Optional `0x / 0X` prefix is accepted.
- Trailing `# comment` text is stripped.

12.2 Binary format (`init_file_format = "bin"`)

Raw binary, dumped to disk as a contiguous byte stream. Each 8 bytes on disk form one 64-bit word; bytes are stored at the matching offset in `data_` without any swap. Matches the `xxd / dd / objcopy -O binary` conventions used by the SMC tooling for `.img` artefacts.

12.3 `auto` resolution

The default `init_file_format = "auto"` picks `bin` if the filename ends in `.img` or `.bin` (case-insensitive), and `hex` otherwise. Any other value falls back to the same auto-detection rule, but the `SC_REPORT_INFO` banner emitted at construction time shows the resolved format so the operator always knows which path was taken.

12.4 Zero-padding rules

File size	Preload result
0 bytes	<code>SC_REPORT_FATAL</code> (empty preload is treated as an error)
<code>< size_bytes</code>	Loaded into <code>data_[0..N-1]</code> ; the remainder is zero
<code>= size_bytes</code>	Exact fit
<code>> size_bytes</code> (hex)	Tolerated iff every overflow word is zero; non-zero overflow word → <code>SC_REPORT_FATAL</code>
<code>> size_bytes</code> (bin)	Truncated silently (binary streams may be padded for tooling)

13. Compliance matrix

Claim	Source	Statement	Model behaviour
C2	<code>test_sep_boot_rom_rw.py::test_read_after_reset</code>	<code>rvalid</code> asserts 1 cycle after a read request	<code>b_transport</code> annotates <code>access_delay_ns</code> (default 1 ns)
C3	<code>test_sep_boot_rom_rw.py::test_preload_readback</code>	Reads return preloaded data (or zero if not preloaded)	Reads return <code>data_[off..off+len-1]</code> verbatim
C4	<code>test_sep_boot_rom_rw.py::test_rom_write_ignored</code>	Writes silently ignored; <code>rvalid</code> still generated	<code>b_transport</code> returns <code>TLM_OK_RESPONSE</code> ; <code>data_</code> unchanged
—	<code>test_sep_boot_rom_preload.py::test_preload_zero_init</code>	Without preload, ROM words are zero	<code>init_file = ""</code> → <code>data_.assign(size, 0)</code>
—	<code>test_sep_boot_rom_preload.py::test_preload_first_words</code>	First 16 words of preload file match <code>data_</code>	Hex / binary parsers both verified by test bench
—	RDL <code>sw=r, hw=r</code>	ROM is software-readable	Production + debug writes both silently discard data
—	RDL <code>memwidth=64</code>	64-bit native word width	Default <code>size_bytes</code> is <code>NUM_ENTRIES × 8</code> ; 8-byte access is the maximum
—	SMC IP convention	All SMC IPs expose <code>rst_n_i</code> even when no state is reset	<code>rst_n_i</code> registered as <code>SC_METHOD</code> ; body is intentionally empty

13.1 Documented deviations

None. All behaviour matches the RDL and the cocotb conformance suite.

13.2 Unmodelled features

Item	Reason
Approximately-Timed (AT) protocol	LT model is sufficient for SW bring-up.
Per-word programmable read latency	Production RTL always returns 1-cycle <code>rvalid</code> ; modelling sub-word latency variation would be misleading at the transaction level.
DMI fast-path	Kept parked behind the same convention as PLIC/CLINT; trivial to add.
Secure-region / key-disable lock	Not part of the current RDL; future revisions may add a CCI flag.

14. Revision history

Version	Date	Author	Notes
1.0	2026-05-26	SMC modelling	Initial release — matches <code>sep_boot_rom.rdl</code> and the SEP cocotb conformance suite (C2/C3/C4).
1.1	2026-06-01	SMC modelling	Reframed as the generic SMC Boot ROM model (SMC CPU-cluster boot path; SEP role-compatible). No behavioural change.

15. Glossary

- **Boot ROM** — A small, preloaded, read-only memory containing the first instructions executed by a processor after reset.
- **Preload** — The act of populating ROM contents at elaboration time from an external file (hex or binary).
- **rvalid** — The AXI4 read-data-valid handshake; the LT model abstracts this into the annotated `b_transport` delay.
- **Sparse-non-zero overflow** — A preload file longer than the target ROM whose overflow region is not entirely zero; treated as a configuration error to prevent silent image-size mismatches.
- **LT (Loosely-Timed)** — TLM-2.0 modelling style: `b_transport`, optional DMI, coarse `sc_time` annotations.
- **RDL** — SystemRDL register-description language; the authoritative Boot ROM declaration lives in `tt-oca-hw/meta/registers/rdl/sep_boot_rom.rdl`.
- **CCI** — SystemC Configuration, Control and Inspection (Accellera cci-1.0.0). Provides the `cci_param<T>` interface used here.
- **SMC** — System Management Controller; the RV64GC management complex whose IP library this model belongs to.
- **SEP** — Secure Enclave Processor; a separate secure core whose boot ROM is structurally identical and is served by the same `smc::bootrom` model with SEP-specific CCI presets.
- **SC_REPORT_FATAL** — SystemC's mechanism for reporting an unrecoverable configuration error during elaboration.

End of document.